

B28: SQL SuperHero Program : Session 32

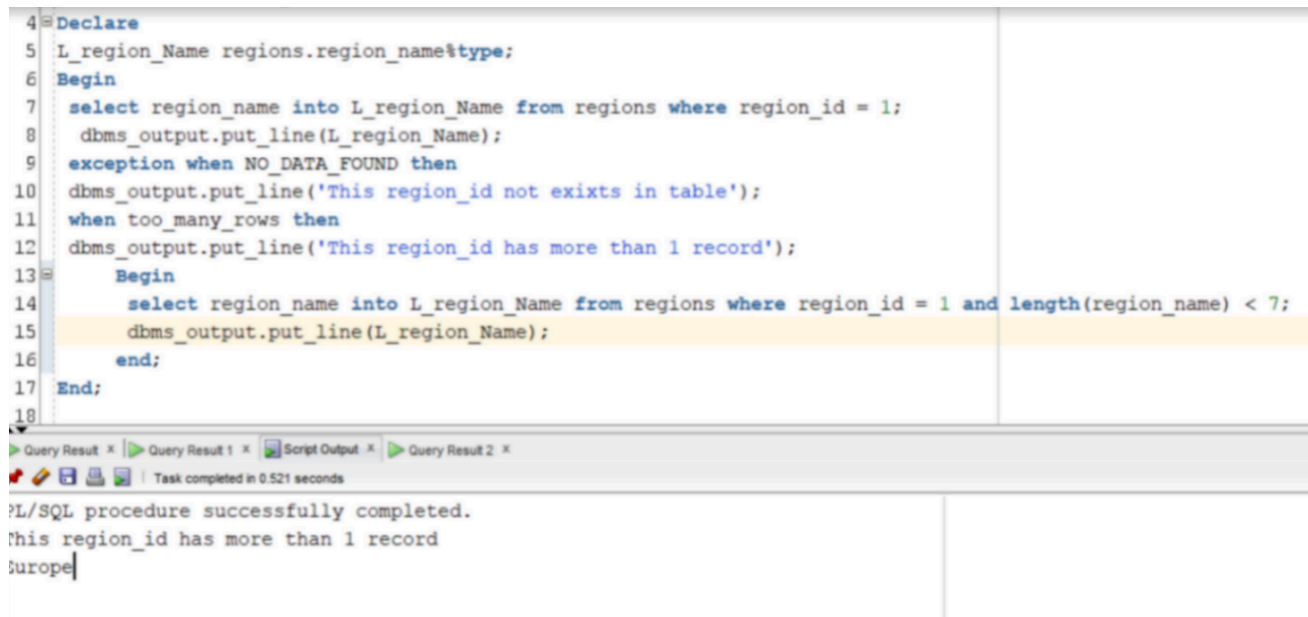
1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

12 April 2025 at 21:33

Exception block can be treated as a new independent/isolated block

```
Declare
L_region_Name regions.region_name%type;
Begin
select region_name into L_region_Name from regions where region_id = 1;
dbms_output.put_line(L_region_Name);
exception when NO_DATA_FOUND then
dbms_output.put_line('This region_id not exists in table');
when too_many_rows then
dbms_output.put_line('This region_id has more than 1 record');
    Begin
        select region_name into L_region_Name from regions where region_id = 1 and length(region_name) < 7;
        dbms_output.put_line(L_region_Name);
    end;
End;
```



```
4 Declare
5 L_region_Name regions.region_name%type;
6 Begin
7 select region_name into L_region_Name from regions where region_id = 1;
8 dbms_output.put_line(L_region_Name);
9 exception when NO_DATA_FOUND then
10 dbms_output.put_line('This region_id not exists in table');
11 when too_many_rows then
12 dbms_output.put_line('This region_id has more than 1 record');
13     Begin
14         select region_name into L_region_Name from regions where region_id = 1 and length(region_name) < 7;
15         dbms_output.put_line(L_region_Name);
16     end;
17 End;
```

Task completed in 0.521 seconds

PL/SQL procedure successfully completed.

This region_id has more than 1 record

Europe

If you see the above code, the table has more than 1 record for region_id = 1, so it gives an exception. control transfer to exception block no_data_found can not handle this.

Go to next exception as too_many_rows, that can handle , because our issue is we have more data than requested

It handles and print the message that is written. we are feel free to write another block here as same like a new block and do the handling this new block can again handle exceptions

Exception within Exception

Yes, we can

```
2 Declare
3 L_region_Name regions.region_name%type;
4 Begin
5 select region_name into L_region_Name from regions where region_id = 1;
6 dbms_output.put_line(L_region_Name);
7 exception when NO_DATA_FOUND then
8 dbms_output.put_line('This region_id not exists in table');
9 when too_many_rows then
10 dbms_output.put_line('This region_id has more than 1 record');
11 Begin
12 select region_name into L_region_Name from regions where region_id = 1 and length(region_name) > 5;
13 dbms_output.put_line(L_region_Name);
14 exception
15 when too_many_rows then
16 dbms_output.put_line('sorry man again too many');
17 end;
18 End;
```

Controller minus:- 5 → 7 → 9 → 10 → 12 → 14

↓ 16

Query Result x Query Result 1 x Script Output x Query Result 2 x

Task completed in 0.569 seconds

PL/SQL procedure successfully completed.

This region_id has more than 1 record

sorry man again too many

we can write Exception within Exception

Can we write a function in Exception/ Can we write a DML in exception

We can do anything. It is an ACID property implementation. Every transaction is independent.
Create a simple function as below for testing:

created a function to test

```
create or replace function get_total_count(p_region_id number) return number as  
l_count number;  
Begin  
  
select count(*) into l_count from regions where region_id = 1;  
return l_count;  
End get_total_count;
```

Function created.

```
declare  
l_name regions.region_name%type;  
x number;  
Begin  
    select region_name into l_name from regions where region_id = 1;  
    dbms_output.put_line(l_name);  
Exception  
    when no_data_found then  
        dbms_output.put_line('Hello');  
    when too_many_rows then  
        dbms_output.put_line('More than 1 record exists');  
        x:= get_total_count(1);  
        dbms_output.put_line('total region exists '|| x);  
    when others then  
        dbms_output.put_line('Hi');  
  
End;
```

Statement processed.
More than 1 record exists
total region exists 2

ACED property

↓
I

```

Declare
L_region_Name hr.regions.region_name%type;
x number;
Begin
  select region_name into L_region_Name from regions where region_id = 1;
  dbms_output.put_line(L_region_Name);
exception when NO_DATA_FOUND then
  dbms_output.put_line('Hello');
when too_many_rows then
  dbms_output.put_line('hello stupid, you entered wrong value');
  x := get_total_count(1);
  dbms_output.put_line('This region has total '||x);
  delete from regions where region_id = 2; → DML here
When others then
  dbms_output.put_line('Hi');

End;

```

```

Statement processed.
hello stupid, you entered wrong value
This region has total 2

```

```

declare
  l_name regions.region_name%type;
  x number;
Begin
  select region_name into l_name from regions where region_id = 1;
Exception → my exception block
  when no_data_found then
    dbms_output.put_line('Hello');
  when too_many_rows then
    dbms_output.put_line('more than 1 record exists');
    x := get_total_count(1); fn called
    dbms_output.put_line('total number of records are '||x);
    delete from regions where region_id = 2; delete
    insert into regions values (55, 'India'); insert
End;

```

Statement processed.
more than 1 record exists
total number of records are 2

✓ → Anything u can do.

=====

====

SQLCODE returns the number of the last encountered error.

SQLERRM returns the message associated with its error-number

The SQLERRM function returns the error message associated with the most recently raised error exception. This function should only be used within the Exception Handling section of your code. SQLCODE and SQLERRM are **Oracle's built-in error reporting functions in PL/SQL**. When an error occurs in PL/SQL at runtime: SQLCODE returns the number of the last encountered error. SQLERRM returns the message associated with its error-number argument.

```
declare
    l_name regions.region_name%Type;
    x number;
Begin
    select region_name into l_name from regions where region_id = 1;
    dbms_output.put_line(l_name);
Exception
    when others then
        dbms_output.put_line(sqlcode);
        dbms_output.put_line(sqlerrm);
End;
```

Statement processed.
-1422
ORA-01422: exact fetch returns more than requested number of rows

Handwritten annotations:

- An orange arrow points from the `sqlcode` argument in the exception handler to the text "Error Code".
- A green arrow points from the `sqlerrm` argument in the exception handler to the text "error message".